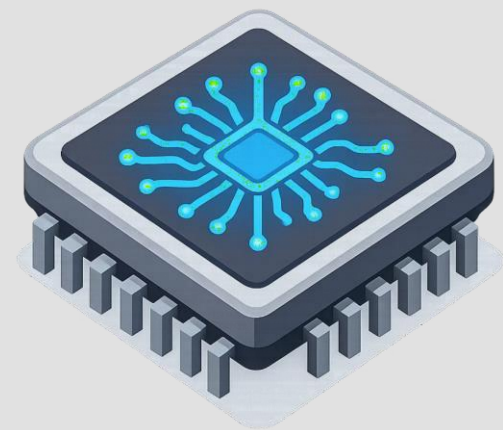


Unidad 2: Procesos e Hilos

El Corazón de la Multitarea



Unidad 2: Procesos

1. *El concepto de proceso.*
2. *Planificación de procesos.*
3. *Operaciones con procesos.*
4. *Procesos cooperativos.*
5. *Hilos de Ejecución (Threads).*
6. *Comunicación entre procesos*

Exploramos cómo los sistemas operativos gestionan la ejecución simultánea de programas, desde el concepto de proceso hasta la comunicación entre ellos.

El Concepto de Proceso

Un **proceso** es un programa en ejecución con su propio espacio de memoria y recursos asignados por el SO.
Es la unidad fundamental de trabajo del sistema.

 **Analogía:** Un proceso es como un chef trabajando en su propia estación: tiene sus ingredientes, utensilios y avanza en su receta de forma independiente.



PID
Identificador único

Estado
Listo · Ejecutando · Bloqueado

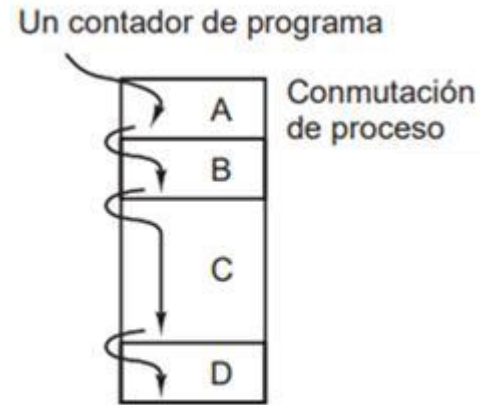
Contexto
PC, registros, pila

Componentes clave

Concepto de Proceso

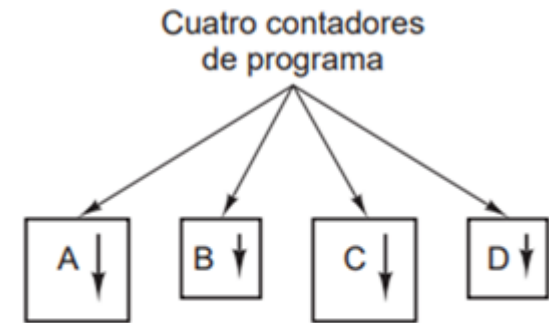
- Un proceso es:
 - Sección de texto (código del programa)
 - Actividad actual, representada por:
 - Valor del contador de programa.
 - Contenido de registros del procesador.
- Además, también incluye:
 - Pila (stack), que contiene datos temporales (parámetros de subrutinas, direcciones de retorno y variables locales).
 - Sección de datos, que contiene variables globales y memoria dinámica.

Modelo del Proceso



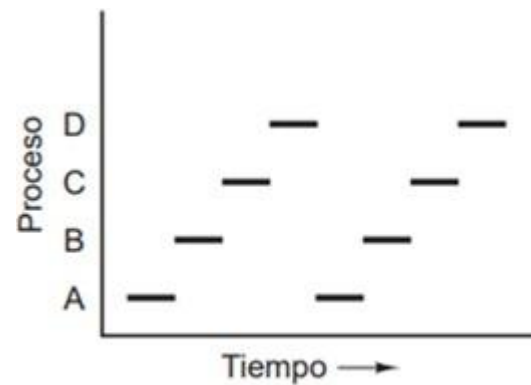
(a)

(a) Multiprogramación de cuatro programas.



(b)

(b) Modelo conceptual de cuatro procesos secuenciales independientes.



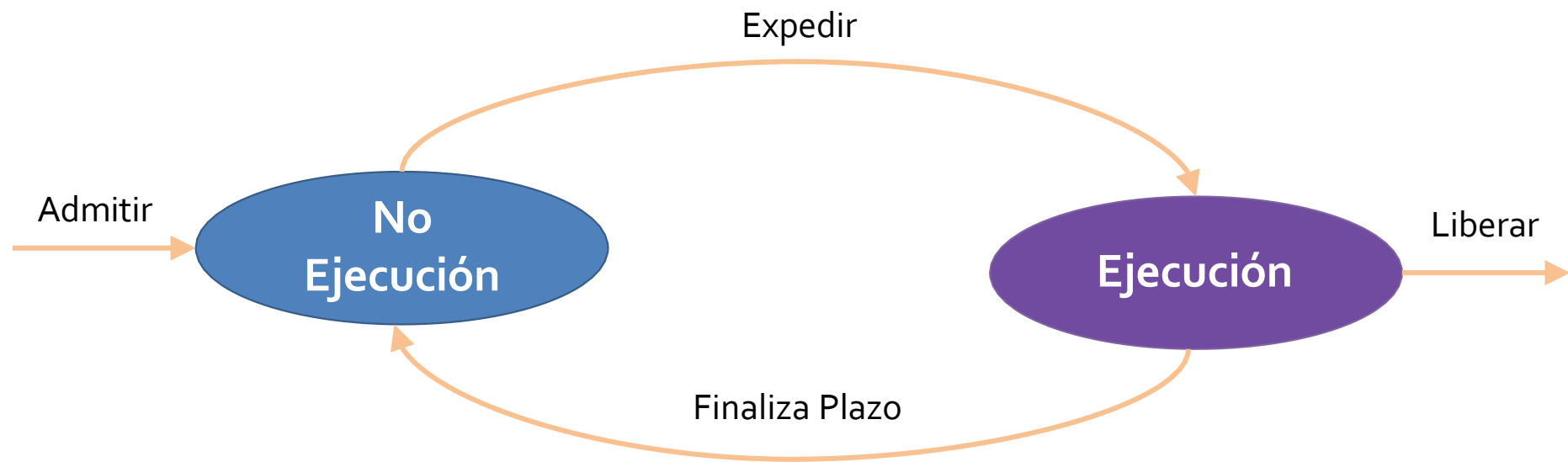
(c)

(c) Sólo hay un programa activo a la vez.

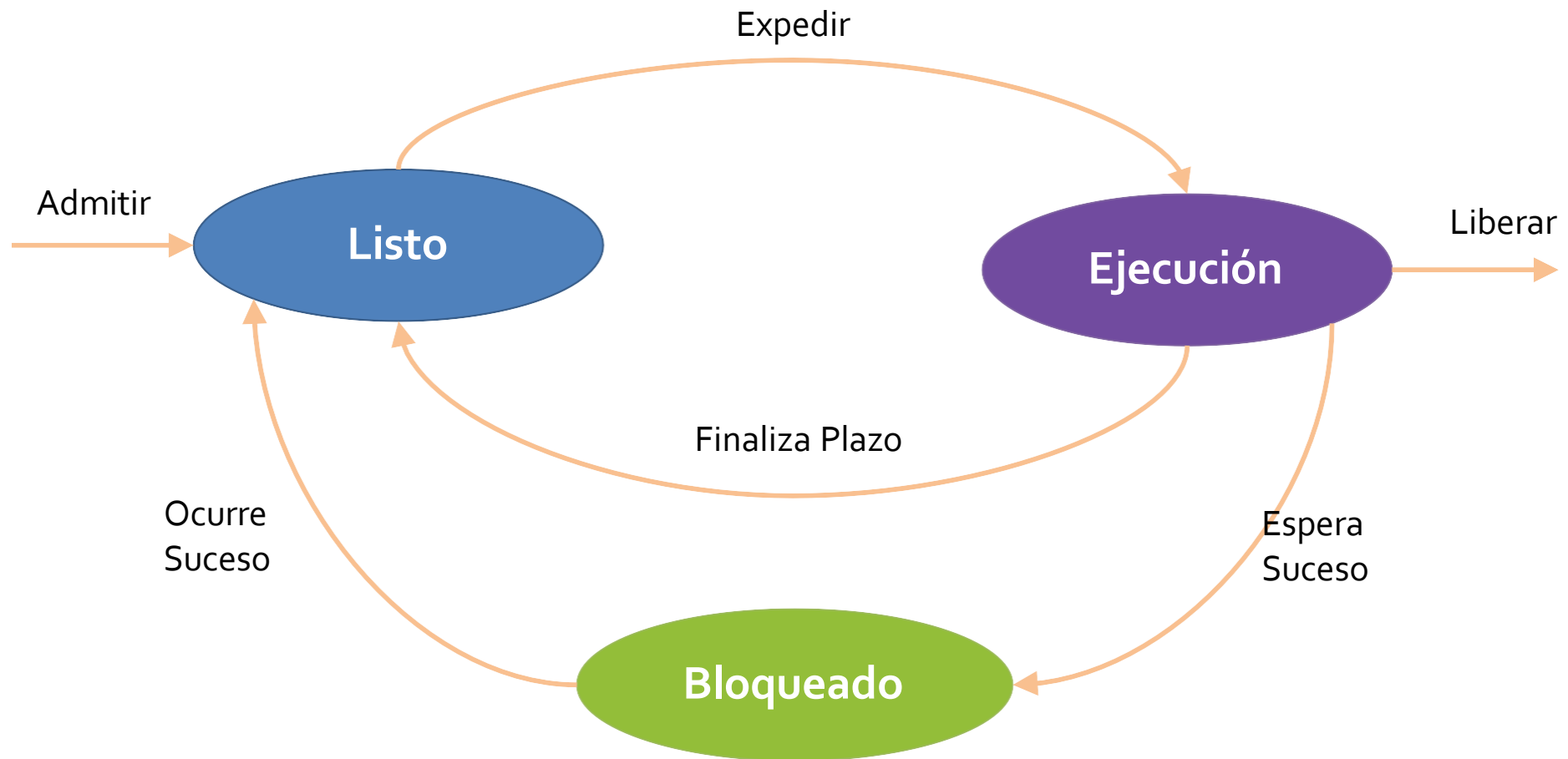
Estados de un Proceso

Un proceso no es simplemente un programa en ejecución, sino una entidad activa que cambia de estado a medida que el sistema operativo gestiona los recursos de la CPU.

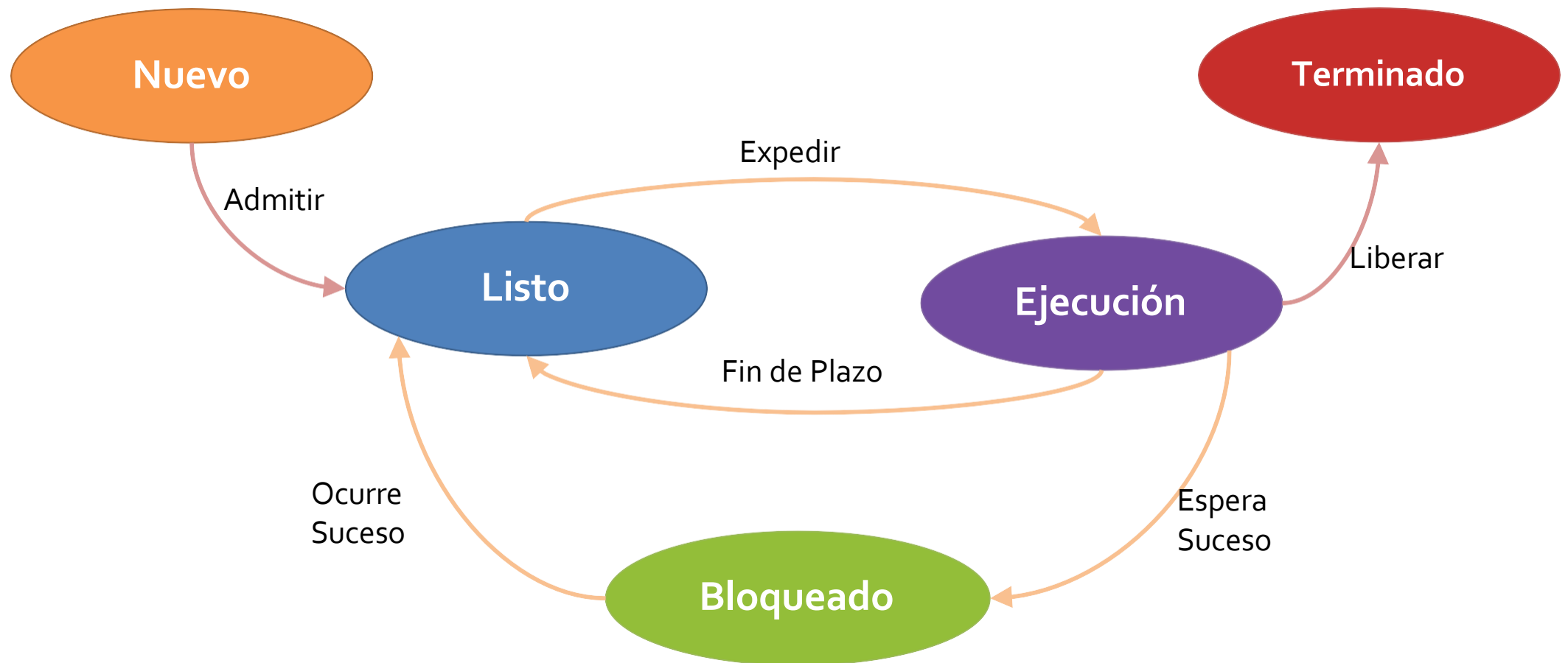
Modelo de 2 estados



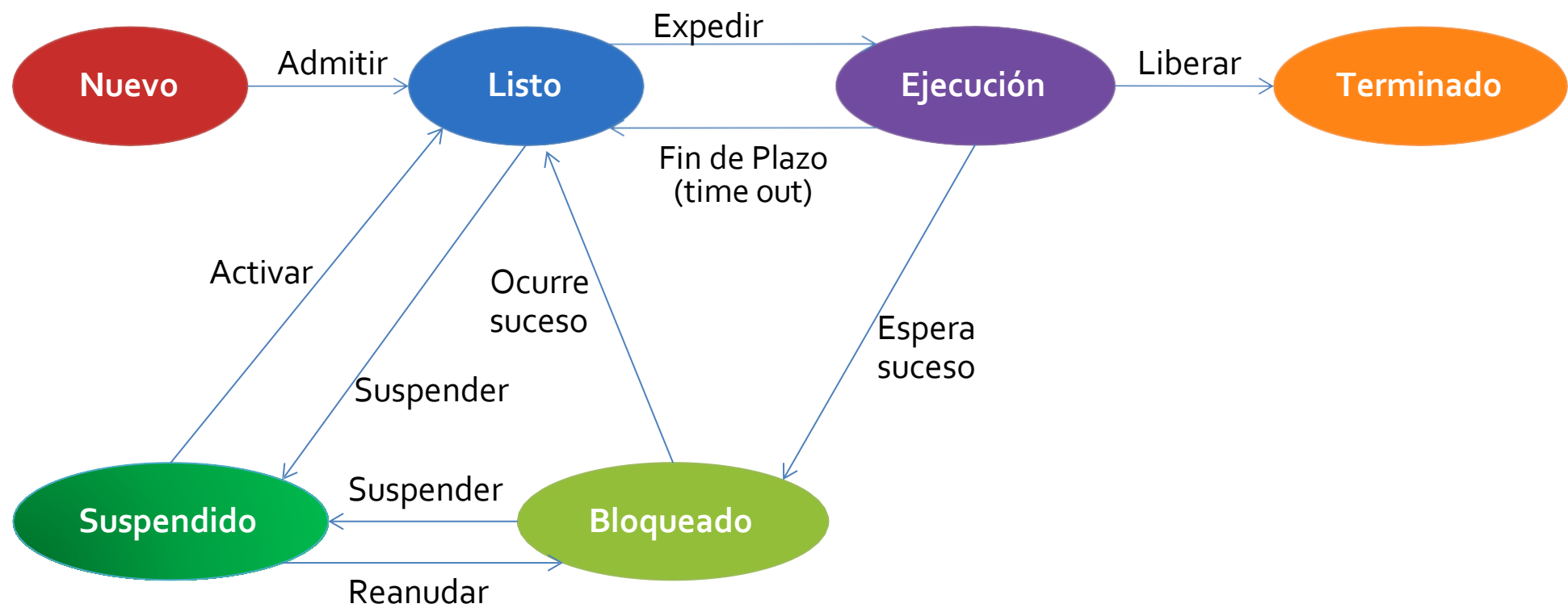
Modelo de 3 estados



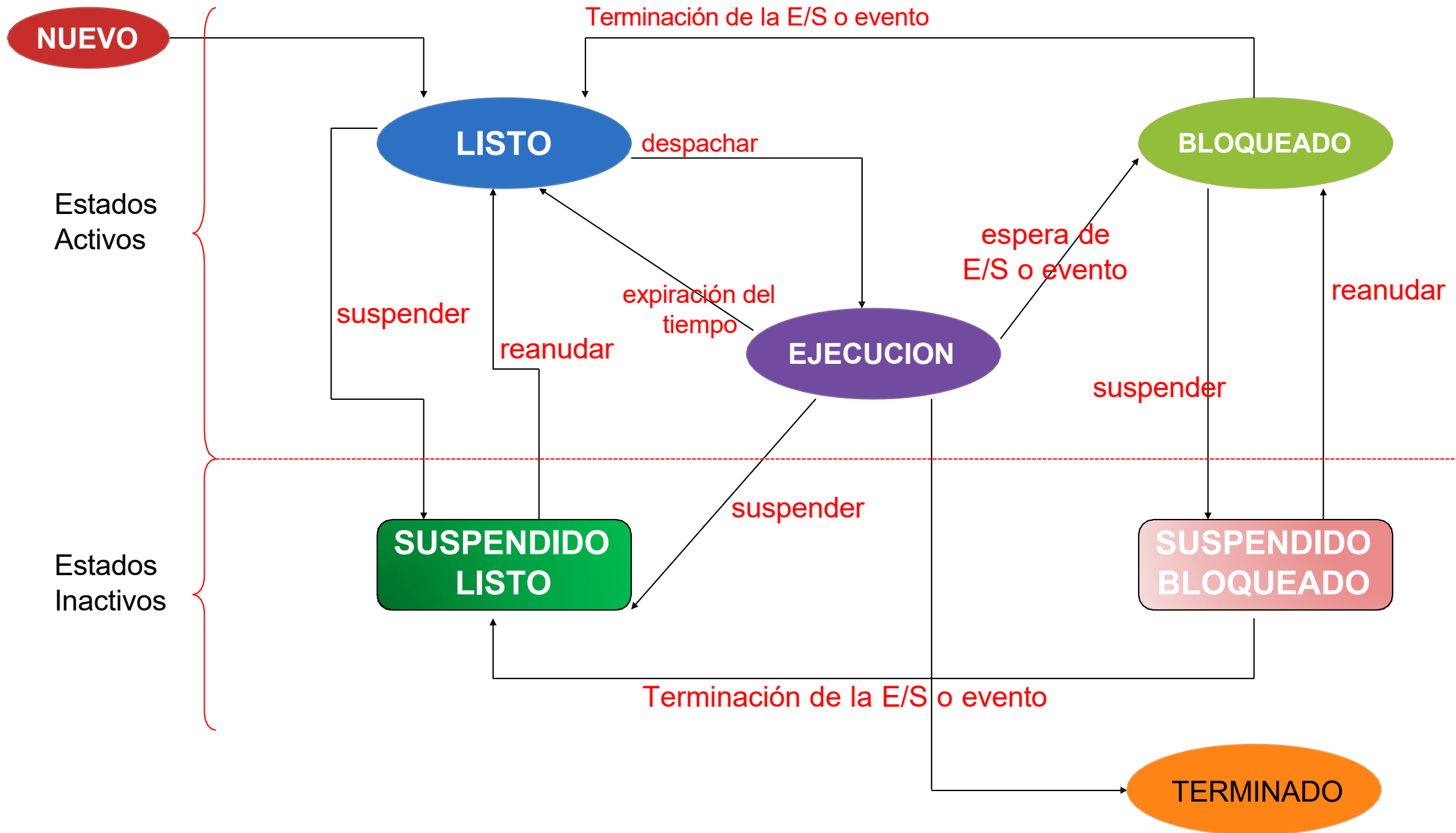
Modelo de 5 estados



Modelo de 6 estados



Modelo de 7 estados



Niveles De Planificación

□ Planificación a largo plazo – planificador de trabajos

- Decide cual será el próximo trabajo que se va a ejecutar, existe en los sistemas de proceso por lotes. Este nivel es el encargado de **crear** los procesos.

□ Planificación a medio plazo – planificador de Swapping

- Es el que decide si un proceso que esta en **ejecución**, **bloqueado** o **suspendido** debe ser extraído de la memoria temporalmente, posteriormente cuando el sistema se encuentre más descargado devolverá dicho proceso a la memoria. Existe en los sistemas de tiempo compartido.

□ Planificación a corto plazo – planificador del procesador

- Es el encargado de decidir cómo y cuando tendrá acceso al procesador un proceso que está **listo** para utilizarlo, lleva a cabo las funciones de multiprogramación, estando siempre residente en memoria.

Planificación de Procesos

- El planificador (scheduler) decide que proceso ocupa la CPU en cada instante, creando la ilusión de multitarea
- Cada algoritmo tiene ventajas según el tipo de carga de trabajo: interactiva, por lotes o de tiempo real.

Cambio de contexto: Guardar el estado del proceso actual y restaurar el del siguiente. Tiene un costo en tiempo de CPU que los algoritmos intentan minimizar.

Operaciones con Procesos

Creación y Terminación

`fork()`

Duplica el proceso padre creando un hijo idéntico

`exec()`

Reemplaza la imagen del proceso por un nuevo programa

Terminación

Salida normal, por error o señal de otro proceso

Operaciones con Procesos

Fenómenos especiales



Proceso Zombie

Terminó pero su padre no leyó su estado de salida. Consume una entrada en la tabla de procesos.



Proceso Huérfano

Su proceso padre terminó antes que él. Es adoptado por `init` (PID 1).

Procesos Cooperativos

No todos los procesos compiten: muchos **colaboran** para alcanzar un objetivo común, compartiendo información y coordinando sus acciones.



Servidores Web

Un proceso maestro crea hijos para atender cada solicitud HTTP de forma independiente y concurrente.



Sistemas Distribuidos

Procesos en distintas máquinas cooperan para procesar grandes volúmenes de datos (ej. MapReduce).

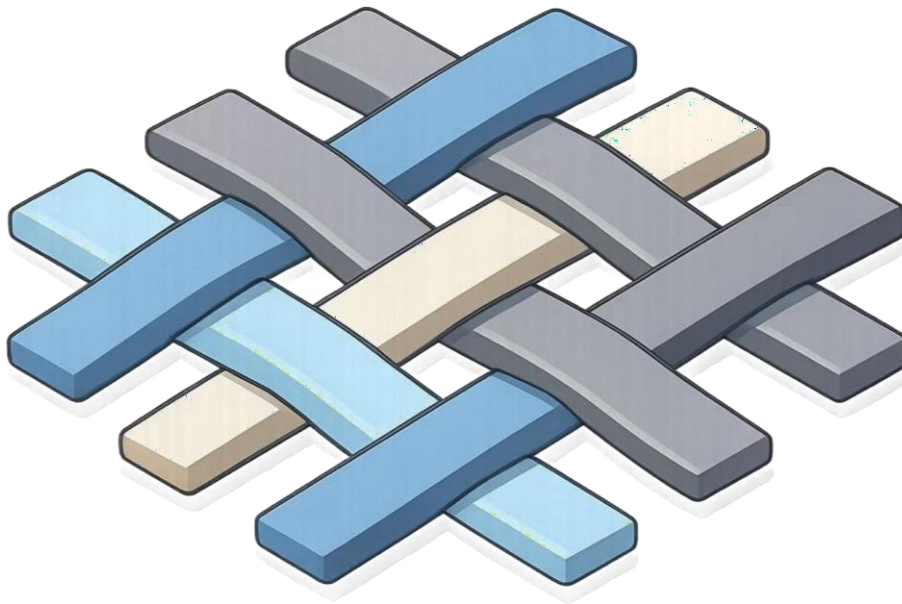


El Gran Desafío

La **sincronización** es crítica: sin ella, surgen condiciones de carrera e inconsistencia de datos.



Hilos de Ejecución (Threads)



Un **hilo** es un flujo de ejecución independiente dentro de un mismo proceso. Todos los hilos de un proceso comparten memoria y recursos.

 **Analogía:** Los hilos son ayudantes del chef en la misma cocina — comparten ingredientes y utensilios, pero cada uno hace su tarea.

Ventajas clave:

- Menor sobrecarga que crear procesos completos
- Comunicación más rápida (memoria compartida)
- Mejor aprovechamiento de CPUs multinúcleo

Tipos de hilos

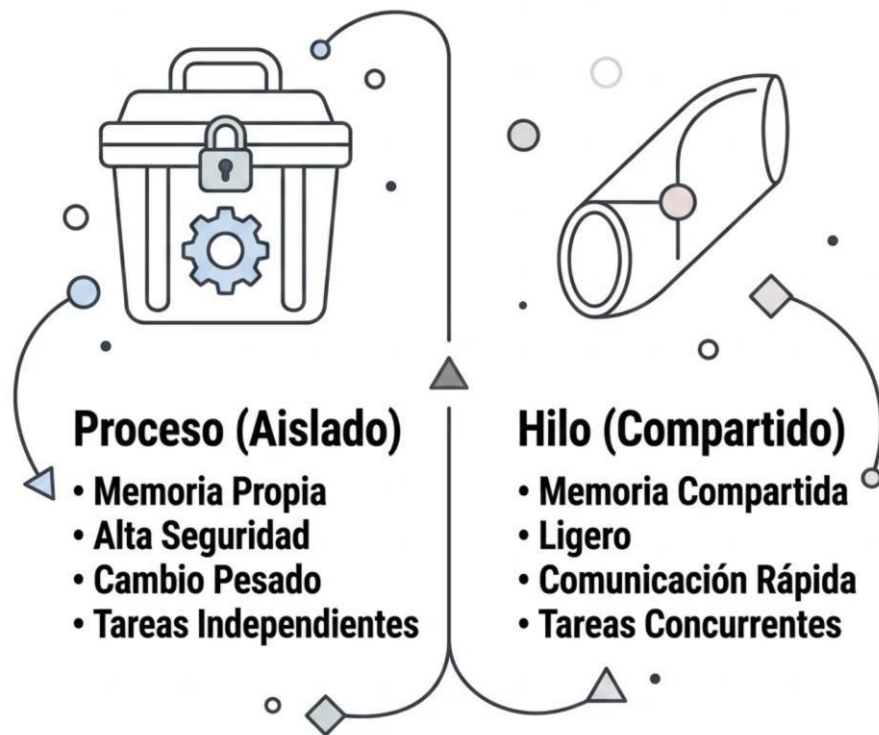
Usuario

Gestionados en espacio de usuario, sin soporte del kernel

Kernel

Gestionados directamente por el SO, mayor control

Hilo vs. Procesos: ¿Cuándo usar cada uno?



Ejemplo real: Navegador Web

Los navegadores modernos combinan ambos enfoques de forma inteligente:

→ Procesos por pestaña

Cada pestaña es un proceso independiente. Si una falla, las demás siguen funcionando.

→ Hilos por tarea

Dentro de cada pestaña, hilos separados descargan imágenes, renderizan HTML y ejecutan JavaScript concurrentemente.

Comunicación entre Procesos (IPC)

Los procesos independientes necesitan mecanismos para intercambiar datos y coordinarse. El SO provee varias formas de IPC:



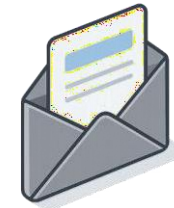
Tuberías (Pipes) y FIFOs

Comunicación unidireccional entre procesos relacionados. Los FIFOs (named pipes) permiten comunicación entre procesos no relacionados.



Memoria Compartida

Región de memoria accesible por múltiples procesos. Es el método **más rápido**, pero requiere sincronización cuidadosa para evitar conflictos.



Paso de Mensajes

Los procesos se envían paquetes de datos estructurados mediante `send()` y `receive()`. Ideal para sistemas distribuidos.